

Lab Programs

17/7/26

Set 7 Customer Demographics Analysis

Dataset: Customer Demographics

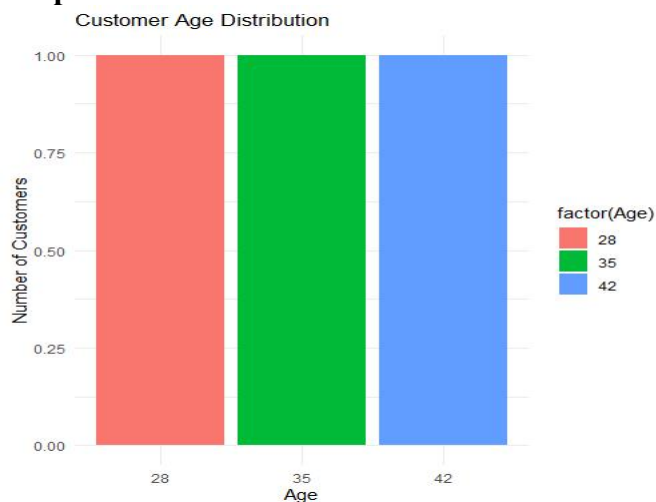
Customer ID	Age	Gender	Income (in \$)
1	28	Female	50000
2	35	Male	60000
3	42	Female	75000

7. A: Create a bar chart to visualize the distribution of customer ages. Label the axes and title the chart.

Code:

```
library(ggplot2)
customer_data <- data.frame(
  CustomerID = c(1,2,3),
  Age = c(28,35,42),
  Gender = c("Female","Male","Female"),
  Income = c(50000,60000,75000)
)
# Bar Chart
ggplot(customer_data,
  aes(x = factor(Age), fill = factor(Age))) +
  geom_bar() +
  labs(
    title = "Customer Age Distribution",
    x = "Age",
    y = "Number of Customers"
  ) +
  theme_minimal()
```

Output:



7.B: Generate a pie chart to represent the distribution of customers by gender.

Code:

```
customer_data <- data.frame(
  Gender = c("Female","Male","Female")
)
```

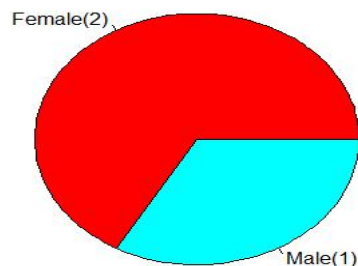
```

gender_count <- table(customer_data$Gender)
# Pie Chart
pie(
  gender_count,
  labels = paste(names(gender_count), "(", gender_count, ")", sep = "" ),
  main = "Customer Gender Distribution",
  col = rainbow(length(gender_count))
)

```

Output:

Customer Gender Distribution



7.C: Build a table to show the demographic information for each customer. Label the table elements.

Code:

```

customer_data <- data.frame(
  CustomerID = c(1,2,3),
  Age = c(28,35,42),
  Gender = c("Female","Male","Female"),
  Income = c(50000,60000,75000)
)
print(customer_data)

```

Output:

CustomerID	Age	Gender	Income
1	28	Female	50000
2	35	Male	60000
3	42	Female	75000

7.D: Develop a Tableau dashboard combining the bar chart, pie chart, and the table for interactive exploration of customer demographics.

Code:

```

# Install once if needed
# install.packages(c("shiny","ggplot2"))
library(shiny)
library(ggplot2)
customer_data <- data.frame(
  CustomerID = c(1,2,3),
  Age = c(28,35,42),
  Gender = c("Female","Male","Female"),
  Income = c(50000,60000,75000)
)
ui <- fluidPage(

```

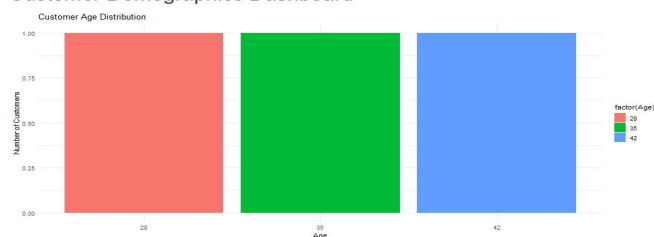
```

titlePanel("Customer Demographics Dashboard"),
fluidRow(
  column(
    6,
    plotOutput("barPlot")
  ),
  column(
    6,
    plotOutput("piePlot")
  )
),
br(),
h3("Customer Demographics Table"),
tableOutput("customerTable")
)
server <- function(input, output){
  output$barPlot <- renderPlot({
    ggplot(customer_data,
      aes(x = factor(Age), fill = factor(Age))) +
    geom_bar() +
    labs(
      title = "Customer Age Distribution",
      x = "Age",
      y = "Number of Customers"
    ) +
    theme_minimal()
  })
  output$piePlot <- renderPlot({
    gender_count <- table(customer_data$Gender)
    pie(
      gender_count,
      labels = names(gender_count),
      main = "Customer Gender Distribution",
      col = rainbow(length(gender_count)))
  })
  output$customerTable <- renderTable({
    customer_data
  })
}
shinyApp(ui, server)

```

Output:

Customer Demographics Dashboard



Customer Gender Distribution



Customer Demographics Table

CustomerID	Age	Gender	Income
1.00	26.00	Female	50000.00
2.00	35.00	Male	60000.00
3.00	42.00	Female	75000.00

Set 8 Employee Performance Analysis

Dataset: Employee Performance

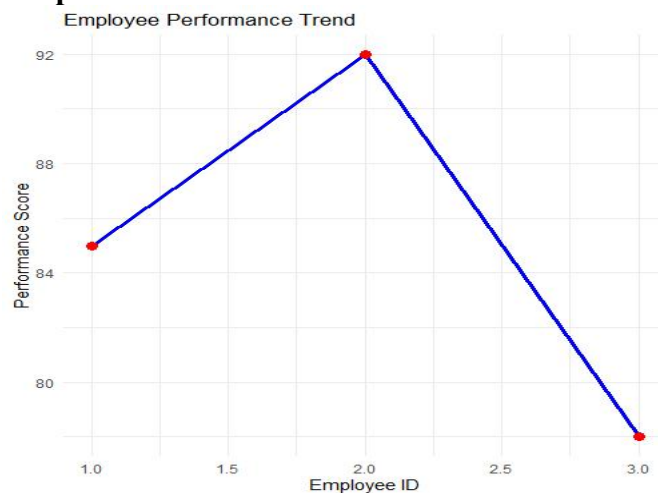
Employee ID	Department	Years of Service	Performance Score
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78

8.A: Create a line chart to visualize the performance trend of employees over time. Label the axes and title the chart

Code:

```
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1,2,3),
  Department = c("Sales","HR","Marketing"),
  YearsService = c(5,3,7),
  Performance = c(85,92,78)
)
ggplot(employee_data,
  aes(x = EmployeeID, y = Performance, group = 1)) +
  geom_line(color = "blue", linewidth = 1.2) +
  geom_point(color = "red", size = 3) +
  labs(
    title = "Employee Performance Trend",
    x = "Employee ID",
    y = "Performance Score"
  ) +
  theme_minimal()
```

Output:



8.B: Generate a bar chart showing the distribution of employees across different departments. Label the chart elements.

Code:

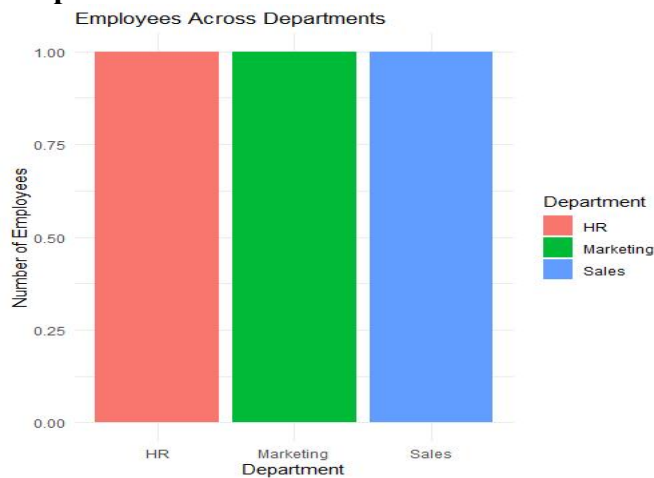
```
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1,2,3),
  Department = c("Sales","HR","Marketing"),
```

```

YearsService = c(5,3,7),
Performance = c(85,92,78)
)
ggplot(employee_data,
  aes(x = Department, fill = Department)) +
  geom_bar() +
  labs(
    title = "Employees Across Departments",
    x = "Department",
    y = "Number of Employees"
  ) +
  theme_minimal()

```

Output:



8.C: Build a table to display the performance data for each employee. Label the table elements.

Code:

```

employee_data <- data.frame(
  EmployeeID = c(1,2,3),
  Department = c("Sales","HR","Marketing"),
  YearsService = c(5,3,7),
  Performance = c(85,92,78)
)
print(employee_data)

```

Output:

EmployeeID	Department	YearsService	Performance
1	1 Sales	5	85
2	2 HR	3	92
3	3 Marketing	7	78

8.D: Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of employee performance data.

Code:

```

library(shiny)
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1,2,3),
  Department = c("Sales","HR","Marketing"),
  YearsService = c(5,3,7),

```

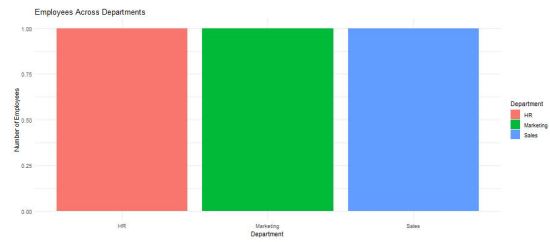
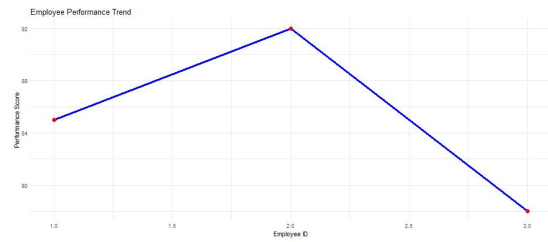
```

    Performance = c(85,92,78)
  )
  ui <- fluidPage(
    titlePanel("Employee Performance Dashboard"),
    fluidRow(
      column(
        6,
        plotOutput("linePlot")
      ),
      column(
        6,
        plotOutput("barPlot")
      )
    ),
    br(),
    h3("Employee Performance Table"),
    tableOutput("employeeTable")
  )
  server <- function(input, output){
    output$linePlot <- renderPlot({
      ggplot(employee_data,
        aes(x = EmployeeID, y = Performance, group = 1)) +
      geom_line(color = "blue", linewidth = 1.2) +
      geom_point(color = "red", size = 3) +
      labs(
        title = "Employee Performance Trend",
        x = "Employee ID",
        y = "Performance Score"
      ) +
      theme_minimal()
    })
    output$barPlot <- renderPlot({
      ggplot(employee_data,
        aes(x = Department, fill = Department)) +
      geom_bar() +
      labs(
        title = "Employees Across Departments",
        x = "Department",
        y = "Number of Employees"
      ) +
      theme_minimal()
    })
    output$employeeTable <- renderTable({
      employee_data
    })
  }
  shinyApp(ui, server)

```

Output:

Employee Performance Dashboard



Employee Performance Table

EmployeeID	Department	Years Service	Performance
1.00	Sales	5.00	85.00
2.00	HR	3.00	92.00
3.00	Marketing	7.00	78.00

set 9

Dataset: Online_Learning_Activity.csv

Student_ID	Gender	Age	Course	Study_Time (hrs)	Videos_Watched	Quiz_Score	Login_Date
L01	Male	20	R	3.5	12	78	2025-01-05
L02	Female	22	R	4.2	15	85	2025-01-05
L03	Male	19	SQL	2.0	8	65	2025-02-08
L04	Female	21	R	5.0	18	92	2025-02-08
L05	Male	23	R	2.5	9	70	2025-03-12
L06	Female	20	SQL	4.0	14	88	2025-03-12

9.A: Import the dataset in R. Plot a histogram for Quiz_Score. Draw a boxplot of Quiz_Score by Course. Add title and labels. Interpret student performance.

Code:

```
library(ggplot2)
```

```
student_data <- data.frame(
```

```
  Student_ID = c("L01","L02","L03","L04","L05","L06"),
```

```
  Gender = c("Male","Female","Male","Female","Male","Female"),
```

```
  Age = c(20,22,19,21,23,20),
```

```
  Course = c("R","R","SQL","R","R","SQL"),
```

```
  Study_Time = c(3.5,4.2,2.0,5.0,2.5,4.0),
```

```
  Videos_Watched = c(12,15,8,18,9,14),
```

```
  Quiz_Score = c(78,85,65,92,70,88),
```

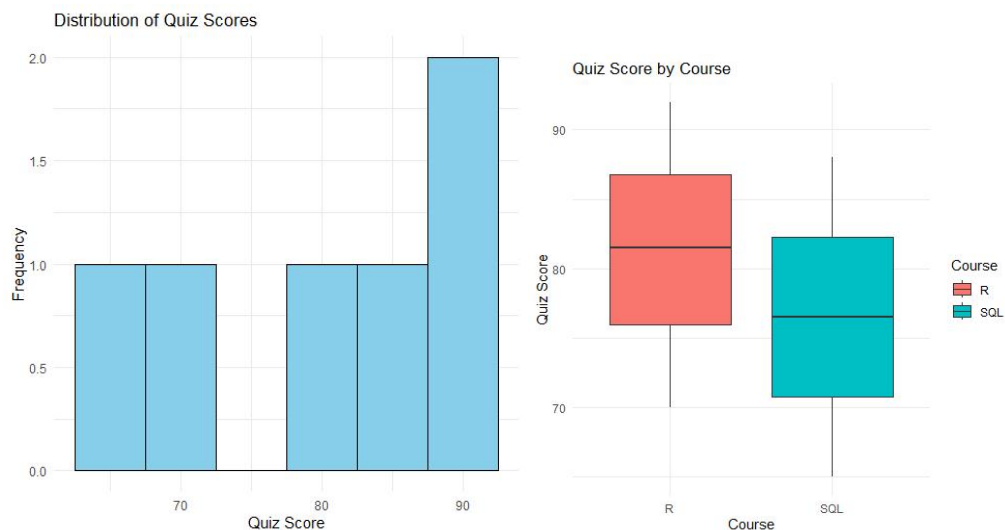
```
  Login_Date = as.Date(c("2025-01-05","2025-01-05","2025-02-08",
```

```
    "2025-02-08","2025-03-12","2025-03-12"))
```

)

```
ggplot(student_data, aes(x = Quiz_Score)) +  
  
  geom_histogram(binwidth = 5, fill = "skyblue", color = "black") +  
  
  labs(title = "Distribution of Quiz Scores",  
  
        x = "Quiz Score",  
  
        y = "Frequency") +  
  
  theme_minimal()  
  
ggplot(student_data, aes(x = Course, y = Quiz_Score, fill = Course)) +  
  
  geom_boxplot() +  
  
  labs(title = "Quiz Score by Course",  
  
        x = "Course",  
  
        y = "Quiz Score") +  
  
  theme_minimal()
```

Output:



9.B:Create a scatter plot using R programming between Study_Time and Quiz_Score. Represent Videos_Watched using bubble size. Apply transparency. Explain the learning pattern.

Code:


```

library(ggplot2)

student_data <- data.frame(

  Student_ID = c("L01","L02","L03","L04","L05","L06"),

  Gender = c("Male","Female","Male","Female","Male","Female"),

  Age = c(20,22,19,21,23,20),

  Course = c("R","R","SQL","R","R","SQL"),

  Study_Time = c(3.5,4.2,2.0,5.0,2.5,4.0),

  Videos_Watched = c(12,15,8,18,9,14),

  Quiz_Score = c(78,85,65,92,70,88))

ggplot(student_data,

  aes(x = Study_Time,

      y = Quiz_Score,

      size = Videos_Watched)) +

  geom_point(color = "blue", alpha = 0.6) +

  labs(title = "Study Time vs Quiz Score",

      x = "Study Time (Hours)",

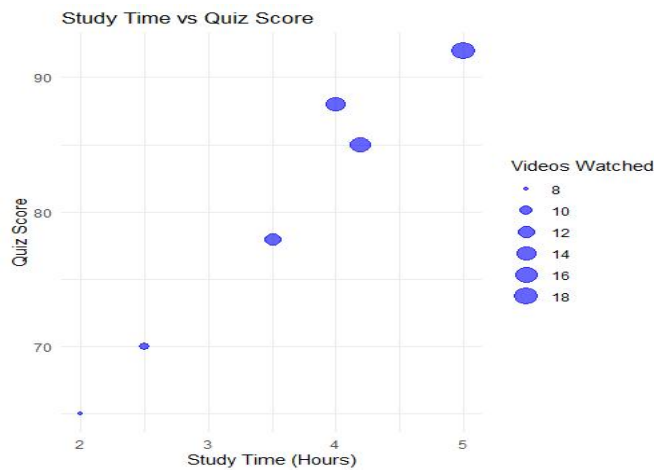
      y = "Quiz Score",

      size = "Videos Watched") +

  theme_minimal()

```

Output:



9.C:Using R programming, convert Login_Date into Date format. Calculate average quiz score per month. Plot a line chart. Apply moving average smoothing. Identify the trend.

Code:

```
library(ggplot2)

monthly_avg <- data.frame(

  Month = c("2025-01", "2025-02", "2025-03"),

  Average_Quiz = c((78+85)/2, (65+92)/2, (70+88)/2)

)monthly_avg$Moving_Average <- c(

  NA,

  mean(monthly_avg$Average_Quiz[1:2]),

  mean(monthly_avg$Average_Quiz[2:3])

)

print(monthly_avg)

ggplot(monthly_avg, aes(x = Month, y = Average_Quiz, group = 1)) +

  geom_line(color = "blue", linewidth = 1) +

  geom_point(color = "blue", size = 3) +

  geom_line(aes(y = Moving_Average),

    color = "red",

    linewidth = 1,
```

```

linetype = "dashed") +

geom_point(aes(y = Moving_Average),

           color = "red",

           size = 3) +

labs(

  title = "Average Monthly Quiz Score",

  x = "Month",

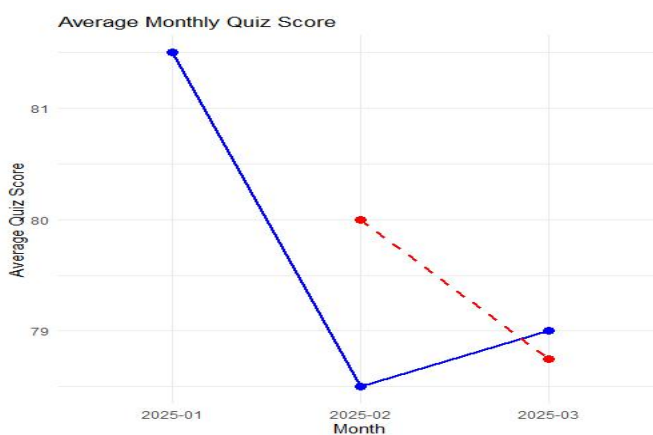
  y = "Average Quiz Score"

) +

theme_minimal()

```

Output:



9.D: Import Online_Learning_Activity.csv into Tableau. Create a bar chart showing average quiz score by Course. Create a scatter plot (Study Time vs Quiz Score). Add filter for Gender and Course. Design a mini interactive dashboard.

Code:

```

library(shiny)

library(ggplot2)

student_data <- data.frame(

  Student_ID = c("L01", "L02", "L03", "L04", "L05", "L06"),

  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),

```

```

Age = c(20,22,19,21,23,20),

Course = c("R","R","SQL","R","R","SQL"),

Study_Time = c(3.5,4.2,2.0,5.0,2.5,4.0),

Videos_Watched = c(12,15,8,18,9,14),

Quiz_Score = c(78,85,65,92,70,88)

)ui <- fluidPage(

  titlePanel("Online Learning Activity Dashboard"),

  sidebarLayout(

    sidebarPanel

      selectInput("gender",

        "Select Gender",

        choices = c("All", unique(student_data$Gender)),

        selected = "All"),

      selectInput("course",

        "Select Course",

        choices = c("All", unique(student_data$Course)),

        selected = "All")),

    mainPanel(

      fluidRow(

        column(6,

          plotOutput("barPlot")),

        column(6,

          plotOutput("scatterPlot"))

      ),

```

```

    br(),

    tableOutput("studentTable"))))

server <- function(input, output){

  filteredData <- reactive({

    df <- student_data

    if(input$gender != "All"){

      df <- df[df$Gender == input$gender, ]

    }

    if(input$course != "All"){

      df <- df[df$Course == input$course, ]

    }

    df

  })

  output$barPlot <- renderPlot({

    avg <- aggregate(Quiz_Score ~ Course,

                     data = filteredData(),

                     mean)

    ggplot(avg,

            aes(x = Course,

                y = Quiz_Score,

                fill = Course)) +

    geom_bar(stat = "identity") +

    labs(

      title = "Average Quiz Score by Course",

      x = "Course",

      y = "Average Quiz Score"

    ) +

```

```

theme_minimal())

output$scatterPlot <- renderPlot({

  ggplot(filteredData(),

    aes(x = Study_Time,

        y = Quiz_Score,

        size = Videos_Watched,

        color = Gender)) +

    geom_point(alpha = 0.7) +

    labs(

      title = "Study Time vs Quiz Score",

      x = "Study Time (Hours)",

      y = "Quiz Score"

    ) +

    theme_minimal())

output$studentTable <- renderTable({

  filteredData()
})

shinyApp(ui = ui, server = server)

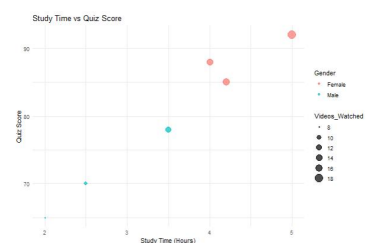
```

Output:

Online Learning Activity Dashboard

Select Gender
All

Select Course
All



Student_ID	Gender	Age	Course	Study_Time	Videos_Watched	Quiz_Score
L01	Male	20.00	R	3.50	12.00	78.00
L02	Female	22.00	R	4.20	15.00	85.00
L03	Male	19.00	SQL	2.00	8.00	65.00
L04	Female	21.00	R	5.00	18.00	92.00
L05	Male	23.00	R	2.50	9.00	70.00
L06	Female	20.00	SQL	4.00	14.00	88.00

Set 10 Survey Responses Analysis

Dataset: Survey Responses

Survey ID	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

10.A: Create a grouped bar chart to visualize the distribution of answers for Question 1. Label the chart elements

Code:

```
library(ggplot2)

survey_data <- data.frame(

  SurveyID = c(1,2,3),

  Question1 = c("A","B","C"),

  Question2 = c("B","A","A"),

  Question3 = c("C","D","B"))

q1_count <- as.data.frame(table(survey_data$Question1))

colnames(q1_count) <- c("Response", "Count")

ggplot(q1_count, aes(x = Response, y = Count, fill = Response)) +

  geom_bar(stat = "identity") +

  labs(

    title = "Distribution of Answers for Question 1",

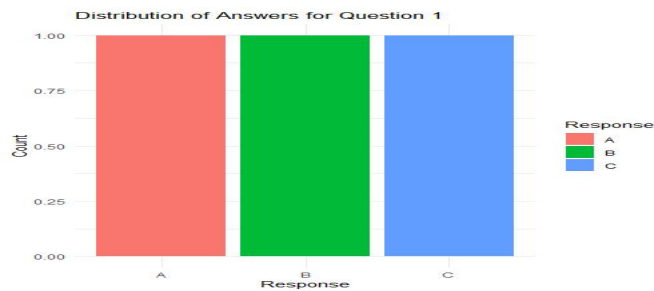
    x = "Response",

    y = "Count"

  ) +

  theme_minimal()
```

Output:



10.B: Generate a stacked bar chart to represent the overall distribution of responses for all three questions.

Code:

```
library(ggplot2)

library(tidyr)

survey_data <- data.frame(

  SurveyID = c(1,2,3),

  Question1 = c("A","B","C"),

  Question2 = c("B","A","A"),

  Question3 = c("C","D","B"))

survey_long <- pivot_longer(

  survey_data,

  cols = c(Question1, Question2, Question3),

  names_to = "Question",

  values_to = "Response")

ggplot(survey_long,

  aes(x = Question, fill = Response)) +

  geom_bar() +

  labs(

    title = "Overall Distribution of Survey Responses",

    x = "Question",
```



```

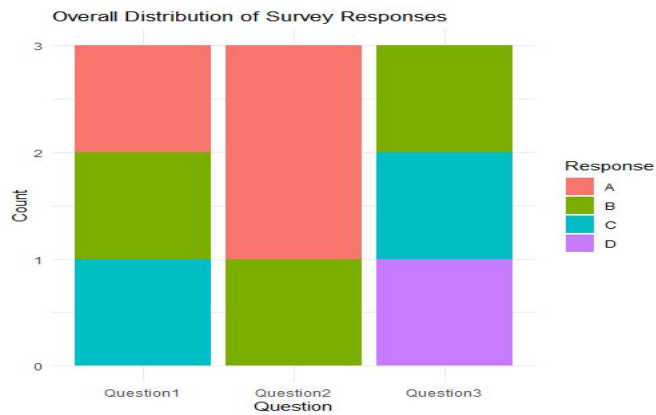
y = "Count"

) +

theme_minimal()

```

Output:



10.C: Build a table to show the survey response data for each survey. Label the table elements.

Code:

```

survey_data <- data.frame(

  SurveyID = c(1,2,3),

  Question1 = c("A","B","C"),

  Question2 = c("B","A","A"),

  Question3 = c("C","D","B"))

print(survey_data)

```

Output:

	SurveyID	Question1	Question2	Question3
1	1	A	B	C
2	2	B	A	D
3	3	C	A	B

10.D: Develop a Tableau dashboard combining the grouped bar chart, stacked bar chart, and the table for interactive exploration of survey responses.

Code:

```
library(shiny)

library(ggplot2)

library(tidyr)

survey_data <- data.frame(

  SurveyID = c(1,2,3),

  Question1 = c("A","B","C"),

  Question2 = c("B","A","A"),

  Question3 = c("C","D","B"))

survey_long <- pivot_longer(

  survey_data,

  cols = c(Question1, Question2, Question3),

  names_to = "Question",

  values_to = "Response")

ui <- fluidPage(

  titlePanel("Survey Responses Dashboard"),

  fluidRow(

    column(6, plotOutput("groupedBar")),

    column(6, plotOutput("stackedBar"))

  ),

  br(),

  h3("Survey Response Table"),

  tableOutput("surveyTable"))

server <- function(input, output){

  output$groupedBar <- renderPlot({
```

```

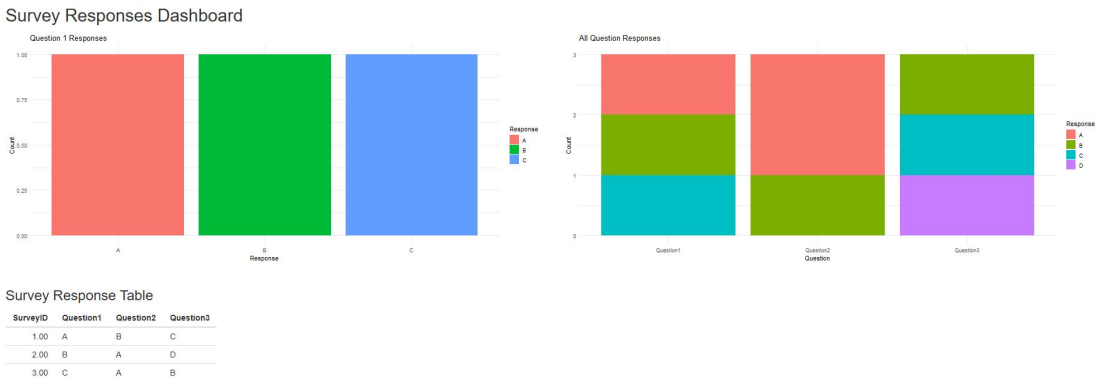
q1_count <- as.data.frame(table(survey_data$Question1))

colnames(q1_count) <- c("Response", "Count")

ggplot(q1_count,
       aes(x = Response, y = Count, fill = Response)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Question 1 Responses",
    x = "Response",
    y = "Count"
  ) +
  theme_minimal()
})output$stackedBar <- renderPlot({
  ggplot(survey_long,
       aes(x = Question, fill = Response)) +
    geom_bar() +
    labs(
      title = "All Question Responses",
      x = "Question",
      y = "Count"
    ) +
    theme_minimal()})
output$surveyTable <- renderTable({
  survey_data
})
}shinyApp(ui, server)

```

Output:



Set 11

Product Category Analysis

Dataset: Product Categories

Category	Sales (in \$)
Electronics	50000
Clothing	35000
Appliances	40000

11.A:Generate a funnel chart to analyze the sales conversion process for each product category. Label the stages and title the chart.

Code:

```
library(plotly)

product_data <- data.frame(

  Category = c("Electronics", "Clothing", "Appliances"),

  Sales = c(50000, 35000, 40000)

)

fig <- plot_ly(

  data = product_data,

  type = "funnel",

  y = ~Category,

  x = ~Sales

)

fig <- layout(
```

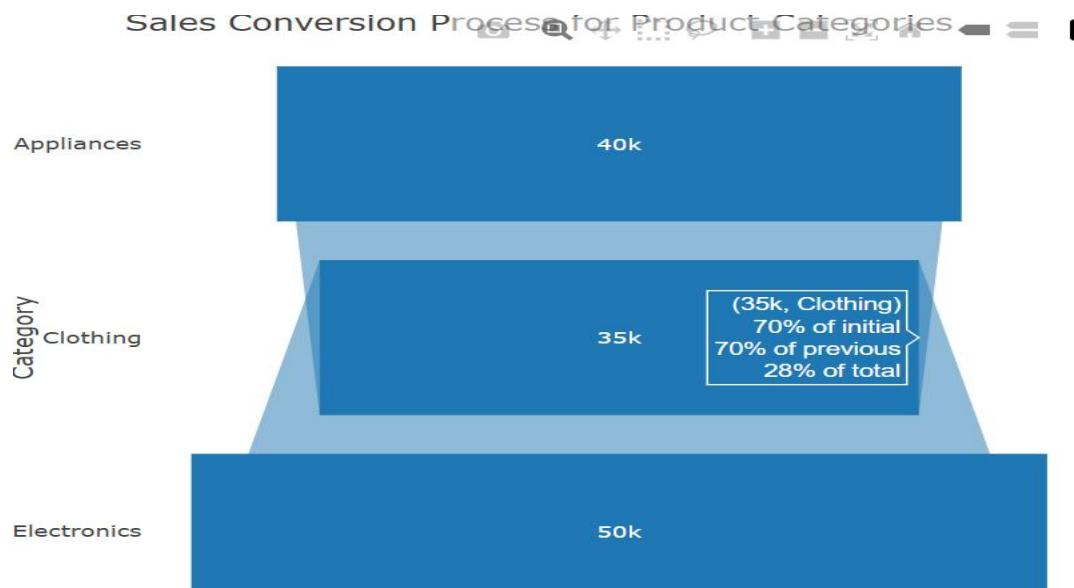
fig,

title = "Sales Conversion Process for Product Categories"

)

fig

Output:



11.B: Build a table to display the sales data for each product category. Label the table elements.

Code:

```
product_data <- data.frame(  
  Category = c("Electronics", "Clothing", "Appliances"),  
  Sales = c(50000, 35000, 40000))  
  
print(product_data)
```

Output:

Category Sales	
1	Electronics 50000
2	Clothing 35000
3	Appliances 40000

11.C: Develop a Tableau dashboard combining the pie chart, funnel chart, and the table for interactive exploration of product category data.

Code:

```
library(shiny)

library(plotly)

product_data <- data.frame(

  Category = c("Electronics", "Clothing", "Appliances"),

  Sales = c(50000, 35000, 40000)

)

ui <- fluidPage(

  titlePanel("Product Category Dashboard"),

  fluidRow(

    column(6,

      plotlyOutput("pieChart")),

    column(6,

      plotlyOutput("funnelChart"))

  ),

  br(),

  h3("Product Category Sales"),

  tableOutput("salesTable")

)

server <- function(input, output){

  output$pieChart <- renderPlotly({

    plot_ly(

      product_data,

      labels = ~Category,
```

```

      values = ~Sales,

      type = "pie"

    ) %>%

    layout(title = "Sales Distribution") })

output$funnelChart <- renderPlotly({

  plot_ly(

    product_data,

    type = "funnel",

    y = ~Category,

    x = ~Sales

  ) %>%

    layout(title = "Sales Funnel"))

output$salesTable <- renderTable({

  product_data

})

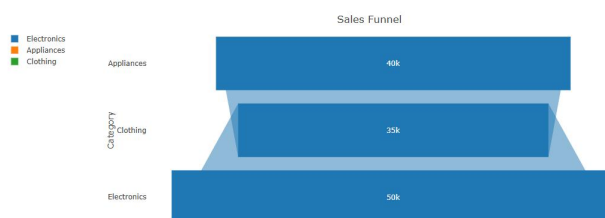
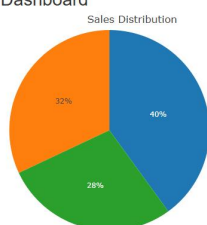
}

```

shinyApp(ui, server)

Output:

Product Category Dashboard



Product Category Sales

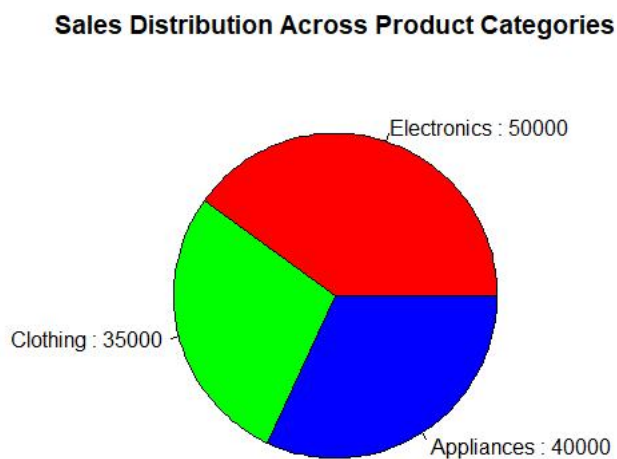
Category	Sales
Electronics	50000.00
Clothing	35000.00
Appliances	40000.00

11.D: Create a pie chart to represent the distribution of sales across product categories. Include labels.

Code:

```
product_data <- data.frame(  
  Category = c("Electronics", "Clothing", "Appliances"),  
  Sales = c(50000, 35000, 40000)  
)  
  
pie(  
  product_data$Sales,  
  labels = paste(product_data$Category, product_data$Sales, sep = " : "),  
  main = "Sales Distribution Across Product Categories",  
  col = rainbow(3)  
)
```

Output:



Set 12 :Dataset: Website Traffic

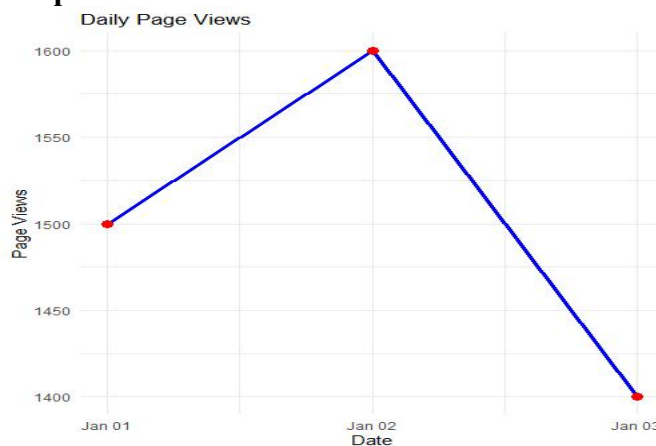
Date	Page Views	Click-through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%

12.A: Create a line chart to visualize the trend in daily page views over time. Label the axes and title the chart.

Code:

```
library(ggplot2)
traffic_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),
  PageViews = c(1500, 1600, 1400),
  CTR = c(2.3, 2.7, 2.0)
)
ggplot(traffic_data,
  aes(x = Date, y = PageViews, group = 1)) +
  geom_line(color = "blue", linewidth = 1.2) +
  geom_point(color = "red", size = 3) +
  labs(
    title = "Daily Page Views",
    x = "Date",
    y = "Page Views"
  ) +
  theme_minimal()
```

Output:

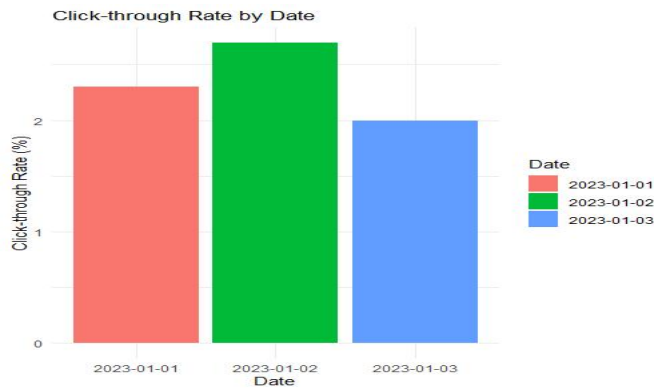


12.B: Generate a bar chart showing the top N days with the highest click-through rates. Label the chart elements.

Code:

```
library(ggplot2)
traffic_data <- data.frame(
  Date = c("2023-01-01", "2023-01-02", "2023-01-03"),
  CTR = c(2.3, 2.7, 2.0)
)
ggplot(traffic_data,
  aes(x = Date, y = CTR, fill = Date)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Click-through Rate by Date",
    x = "Date",
    y = "Click-through Rate (%)"
  ) +
  theme_minimal()
```

Output:



12.C: Build a table to show the website traffic data for each day. Label the table elements.

Code:

```
traffic_data <- data.frame(
  Date = c("2023-01-01", "2023-01-02", "2023-01-03"),
  PageViews = c(1500, 1600, 1400),
  CTR = c("2.3%", "2.7%", "2.0%")
)
print(traffic_data)
```

Output:

	Date	PageViews	CTR
1	2023-01-01	1500	2.3%
2	2023-01-02	1600	2.7%
3	2023-01-03	1400	2.0%

12.D: Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of website traffic data.

Code:

```
library(shiny)
library(ggplot2)
traffic_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),
  PageViews = c(1500, 1600, 1400),
  CTR = c(2.3, 2.7, 2.0)
)
ui <- fluidPage(
  titlePanel("Website Traffic Dashboard"),
  fluidRow(
    column(6,
      plotOutput("linePlot"),

      column(6,
        plotOutput("barPlot")
      ),
    ),
  br(),
  h3("Website Traffic Data"),
  tableOutput("trafficTable")
)
```

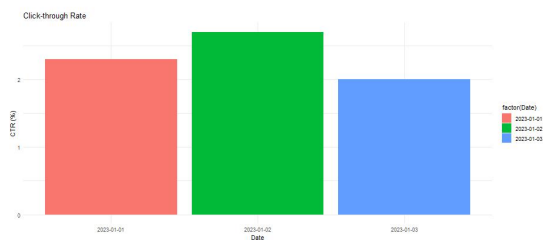
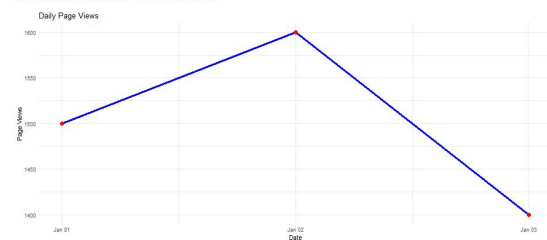
```

server <- function(input, output){
  output$linePlot <- renderPlot({
    ggplot(traffic_data,
      aes(x = Date, y = PageViews, group = 1)) +
    geom_line(color = "blue", linewidth = 1.2) +
    geom_point(color = "red", size = 3) +
    labs(
      title = "Daily Page Views",
      x = "Date",
      y = "Page Views"
    ) +
    theme_minimal()
  })
  output$barPlot <- renderPlot({
    ggplot(traffic_data,
      aes(x = factor(Date), y = CTR, fill = factor(Date))) +
    geom_bar(stat = "identity") +
    labs(
      title = "Click-through Rate",
      x = "Date",
      y = "CTR (%)"
    ) +
    theme_minimal()
  })
  output$trafficTable <- renderTable({
    traffic_data
  })
}
shinyApp(ui, server)

```

Output:

Website Traffic Dashboard



Website Traffic Data

Date	PageViews	CTR
193558.00	1500.00	2.30
193559.00	1600.00	2.70
193560.00	1400.00	2.00

